



Faculty of Science



Scientific Computing

Nonlinear Equations 2: Multi-dimensional

Faculty of Science
University of Copenhagen



5. Nonlinear Equations

Topics today:

- Section 5.6: *Multi-dimensional Nonlinear Equations*
 - Newton's method
 - *Broyden's method*
 - ... + Sherman-Morrison
- Examples and Exercises (optional)
- *Assignment Hints*



But first:

Perhaps the most important competence to pick up from the lessons this week (and next week):

- ❑ While we have access to excellent libraries with standard procedures, understanding what goes on inside still important
- ❑ Even before that: the details of how one formulates a problem (e.g. the form of the $y(x) = 0$ to solve for) are very important
 - Often referred to as pre-conditioning
- ❑ In the Assignment we have, in principle, 7 equations and 7 unknowns ... ! But we are able to make 1-dimensional
 - An example of algebraic simplification



Nonlinear Systems of Equations = Multi-dimensional (N-dim) non-linear eq

Reminder: nonlinear systems of equations are of the type

$$\mathbf{f}(\mathbf{x}) = \mathbf{0}$$

where $\mathbf{f}(\mathbf{x})$ in general is a mapping of a set of real numbers

$$\mathbf{x} = [x_1 \ x_2 \ \dots \ x_n]^T$$

to another set of real numbers

$$\mathbf{f} = [f_1(x_1, x_2, \dots, x_n) \ f_2() \ \dots \ f_n()]^T$$

Seek values of n arguments that make all n function values zero.



Essential meta-topics today

- Importance of simplifying, when possible
- Importance of understanding the methods
(even if one has libraries available)
- Importance of cheating, in some cases ;-)
(will be explained ...)



Multi-dimensional vs. one-dimensional

The multi-dimensional case is much more difficult, because

- A wider variety of behavior is possible, so determining existence and number of solutions, or good starting values, is more complex
- In general, cannot guarantee convergence, or bracket solutions to produce absolutely safe methods
- Computational overhead increases with the dimensionality of the problem

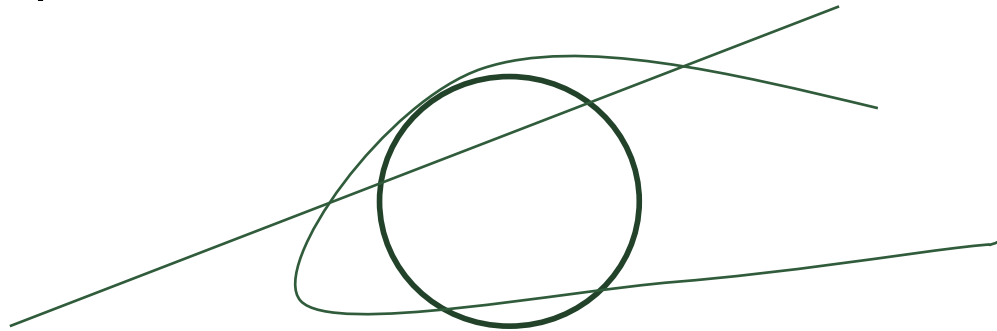


5.2: Existence and Uniqueness (reminder)

Each equation

$$f_i(x_1, x_2, \dots, x_n) = 0$$

corresponds to a hyper-surface in x -space of dimension $(n-1)$, and each additional equation corresponds to intersections of hypersurfaces, finally ending up with (perhaps!) a point – or many points – or no point at all!



Curved surfaces can intersect in many ways, so there can be any number of solutions!

Multi-dimensional method – overview

To make it easier to follow the subsequent discussion, here's an overview:

1. The Newton's method could generalize to N-dim!"
 - Sure, but computing derivatives and inverting matrices is expensive!
2. "Ok, so let's generalize the secant method!"
 - Yeah, great – no derivatives, but still inversion!
3. "No, not if we're smart, and use Broyden's method, with a Sherman-Morrison update!"
 - OK?



Taylor Expansion

For a multi-dimensional function that vanishes at $\mathbf{x}$, Taylor expansion gives

$$F(\mathbf{x}^{(k)}) = 0 + F'(\mathbf{x}^{(k)})(\mathbf{x}^{(k)} - \mathbf{x}) + \dots$$

where $F'(\mathbf{x})$ is a matrix with partial derivatives – the Jacobian matrix $\partial f_i / \partial x_j$.

This suggests a generalization of Newton's method for one-dimensional nonlinear equations to multi-dimensional nonlinear equations

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - F'(\mathbf{x}^{(k)})^{-1} F(\mathbf{x}^{(k)})$$



Newton's Method

Newton's method for systems of equations is a generalization of the one-dimensional method, where the derivative of the 1-D function is replaced by the Jacobian (derivative matrix) of the 3-D function.

$$x^{(k+1)} = x^{(k)} - F'(x^{(k)})^{-1} F(x^{(k)})$$

- As in 1-D, the final convergence is quadratic, if the function is well behaved (non-singular Jacobian).
- Apparently, this requires computing and inverting a Jacobian matrix in every step, but this *can be avoided*, using generalizations of the secant methods from 1-D to n-D.
- The most common of these methods is Broyden's method



Newton Expanded

$$f_i(x_1, x_2, \dots, x_n) = 0$$

$$f_i(x_1 + h_1, x_2 + h_2, \dots, x_n + h_n) = f_i(x_1, x_2, \dots, x_n) + h_1 \frac{\delta f_i}{\delta x_1} + h_2 \frac{\delta f_i}{\delta x_2} + \dots + h_n \frac{\delta f_i}{\delta x_n}$$

We are solving for 0 so:

$$h_1 \frac{\delta f_i}{\delta x_1} + h_2 \frac{\delta f_i}{\delta x_2} + \dots + h_n \frac{\delta f_i}{\delta x_n} = -f_i(x_1, x_2, \dots, x_n)$$

So updating a set of guesses reduces to solving a set of linear equations



Code

```
from numpy import array
from numpy.linalg import solve

def f(x):
    return array( [(x[0] + 2*x[1] - 2), (x[0]**2 + 4 * x[1]**2 - 4)] )

def Jf(x):
    return array( [ [1, 2], [2*x[0], 8*x[1]] ] )

def newton(f, Jd, x):
    for _ in range(20):
        s = solve( Jf( x ), -1 * f( x ) )
        x += s
    return x

result = newton( f, Jf, array( [1.0, 2.0]) )
print( result )
```

```
[1.50295992e-16 1.00000000e+00]
```



Secant Updating Methods

Secant updating methods *increase efficiency* by

- Using function values at successive iterates to build an *approximate Jacobian* and avoiding explicit evaluation of derivatives
- *Updating the factorization* of an approximate Jacobian rather than refactoring in each iteration

Most secant updating methods have *superlinear but not quadratic* convergence rate, just as in one-dimension



Broyden's Method – the general idea

Let $\mathbf{B}_k$ be an approximation of $\mathbf{F}'(\mathbf{x}^{(k)})$ – the Jacobian matrix of the multi-dimensional function $\mathbf{F}(\mathbf{x})$ at the point $\mathbf{x}=\mathbf{x}^{(k)}$.

A Newton-like update step, with a correction: $\mathbf{h}_k$ is

$$\mathbf{B}_k \mathbf{h}_k = -\mathbf{F}(\mathbf{x}^{(k)}); \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{h}_k$$

Taylor's formula is

$$\mathbf{y}_k = \mathbf{F}(\mathbf{x}^{(k+1)}) - \mathbf{F}(\mathbf{x}^{(k)}) = \mathbf{h}_k \mathbf{F}'(\mathbf{x}^{(k)}) + \dots$$

Define $\mathbf{B}_{k+1}$ by

$$\mathbf{B}_{k+1} \mathbf{h}_k = \mathbf{y}_k, \text{ and } \mathbf{B}_{k+1} \mathbf{z} = \mathbf{B}_k \mathbf{z} \text{ for all } \mathbf{z} \text{ with } \mathbf{h}_k^T \mathbf{z} = 0$$

So, $\mathbf{B}_{k+1}$ gives the "right" correction, and is "otherwise" like $\mathbf{B}_k$



Broyden's Method – how it's constructed

The requirements on B_{k+1} are

$$B_{k+1} h_k = y_k, \text{ and } B_{k+1} z = B_k z \text{ for all } z \text{ with } h_k^\top z = 0$$

The idea here is that the first requirement gives the right update, while the second makes sure the new matrix is consistent with the previous correction steps.

By construction, this is fulfilled by

$$B_{k+1} = B_k + [(y_k - B_k h_k) h_k^\top] / (h_k^\top h_k)$$

Proof? “Just” multiply from the right with h_k !



Broyden's Method – the fancy part

The update of the inverse $H_{k+1} = B_{k+1}^{-1}$ can be computed more inexpensively with the 'Sherman-Morrison' formula:

$$(A + uv^T)^{-1} = A^{-1} - A^{-1}uv^TA^{-1}/(1 + v^TA^{-1}u)$$

This is an identity (can be proven by matrix manipulations – many!)

Applied to what we want to do in Broyden's method, it gives

$$H_{k+1} = H_k - [(H_k y_k - h_k) (h_k^T H_k)] / ((h_k^T H_k) y_k)$$

So, with this trick one can *update the inverse of a matrix* (which was never really inverted in the first place)



The Gradient Method

An alternative to the Newton direction is to use the *gradient direction*. The gradient direction is the "steepest descent" direction for the 2-norm of the function values:

$$d_k = -\alpha_k \nabla \phi = -\alpha_k (\nabla F)^T F$$

where the right hand side contains the matrix product of the transpose of the Jacobian (derivative) matrix times the vector function value.

The step size factor α_k is chosen from a linearization of the problem of minimizing the 2-norm of the function values.

The gradient method is *generally better* at reducing the function values *in the first phase of the iterations* while the Newton method is faster during final convergence.



ASSIGNMENT 3



Lennard Jones Potentials

Overly simplified particle dynamics model

- An attractive force
- A repulsive force

$$V_{LJ}(r) = \frac{A}{r^{12}} - \frac{B}{r^6}$$

$V_{LJ}(r)$ is the potential at distance r between two atoms

$$A = 4\epsilon\sigma^{12}$$

$$B = 4\epsilon\sigma^6$$

ϵ is the minimal potential between two atoms.

σ is the distance where V is zero.

- We work with Argon only



Questions

- 1 Write a function that finds the potential between a set of atoms
 - Pay attention to evaluation speed
- 2 Write a solver to find roots in 1D systems
 - Demonstrate with two argon atoms
- 3 Write a CG solver to find the minimum potential of a set of atoms
- 4 Write a heuristic solver to find the minimum potential

